Failure Modes in LLM-Generated Z3 Proofs: A Taxonomy and Audit Methodology

Vishnu Kchitti
vishnu@arbiter.dev

April 2026

Abstract

Z3-based LLM theorem proving pipelines report high solve rates, but the encodings are rarely inspected. We audited proofs generated by our own pipeline—manual inspection of 23 hard-tier miniF2F problems, semi-automated vacuity testing of 50 sampled proofs (LLM-assisted classification), and certificate classification of 54 FormalMATH results—and find that **70% of Z3-only proofs are invalid** (74% including partial proofs). Our reported solve rate of 83% collapses to approximately 26% on the full test set and 13% on the hard-tier subset after auditing. We attempted to fix this using Knuckledragger proof certificates. Certificates prevent one failure mode by construction: assumed-conclusion failures, where the conclusion is directly asserted as a premise (2% of Z3-only failures). They do not eliminate vacuous proofs arising from contradictory premises (40% of failures), which still pass through the certificate pipeline. A deeper problem remains: our pipeline responds to certificate requirements by proving trivial lemmas (`kd.prove(x <= x)`) rather than decomposing the actual theorem, yielding 46.3% trivial certificates on FormalMATH (a different benchmark tested under a different pipeline—results are not directly comparable to the miniF2F numbers). We hypothesize that a key bottleneck is theorem decomposition—our pipeline appears unable to break hard theorems into formal sub-lemmas that map faithfully to the original claim, though we have not isolated this from other confounds such as prompt design or model capability limits. We release our audit corpus and failure taxonomy as a methodology for evaluating the integrity of Z3-based proof pipelines. Note: leading theorem proving systems (Seed-Prover, HILBERT, AlphaProof) use Lean 4, where the proof assistant itself enforces correctness; these findings apply specifically to Z3-based approaches where encoding fidelity is not automatically checked.

1 Introduction

The miniF2F benchmark [1] has become the de facto leaderboard for LLM-based formal theorem proving. Recent systems report extraordinary numbers: Seed-Prover achieves ∼100% on the test split [2], HILBERT reaches 99.2% [3], and Kimina-Prover reports 92.2% [4]. These results are celebrated as evidence that LLMs can perform genuine mathematical reasoning.

Leading systems (Seed-Prover, HILBERT, AlphaProof) use Lean 4, where the proof assistant's type checker enforces correctness by construction—a Lean proof that compiles is a genuine proof. But for SMT-based approaches—where an LLM generates a Z3 script and the solver returns `unsat`—correctness depends entirely on encoding fidelity, which is external to the solver. When Z3 returns `unsat`, it means the negated formula is unsatisfiable. It says nothing about whether the encoding faithfully represents the mathematical claim. An LLM can assert the conclusion as a premise, encode a weaker statement, or check only trivial arithmetic. Z3 returns `unsat` in all these cases.

We conducted three complementary audits of Z3 proofs generated by our own pipeline—each targeting a different failure surface—and found the same pattern every time: reported solve rates are dramatically inflated by encoding failures.

- **Manual audit** (23 hard-tier miniF2F): 13% valid, 52% invalid.
- **Semi-automated audit** (50-sample full miniF2F, Z3-only): 26% valid, 4% partial, 70% invalid—including 40% vacuous (LLM-assisted classification; see §2).
- **Certificate audit** (54 FormalMATH results, Knuckledragger pipeline): 7.4% genuinely valid, 46.3% trivial certificates. (Note: FormalMATH is a different benchmark tested under a different pipeline; these numbers are not directly comparable to the miniF2F results above.)

We then tried to fix the problem by introducing Knuckledragger proof certificates [9]—tamper-proof `Proof` objects that prevent conclusions from being assumed as premises. The certificates work for what they target: they eliminate the entire class of vacuous proofs that constitutes 40% of Z3-only failures. But the remaining failures reveal a deeper problem. LLMs respond to the certificate requirement by proving trivial lemmas (`kd.prove(x <= x)`) and wrapping them in valid-looking certificate chains. The `Proof` object is genuine—it just does not prove the theorem.

The root cause is not the proof backend. Our best hypothesis is that our pipeline fails at theorem decomposition—it proves what is easy (trivial arithmetic, tautologies) and skips what is hard (the actual mathematical content). We have not definitively isolated decomposition failure from other confounds (prompt design, model capacity, benchmark-specific parsing), but the pattern is consistent across all three audits. We do not claim this generalizes to all LLMs or pipelines; it describes our specific system (gpt-5.4-mini, 3 retries, default Z3 timeout).

This paper presents an honest account: we found a serious problem, tried to fix it, and the fix is partial. We report what works, what does not, and what the field still needs.

Contributions.

1. Three complementary audits of our own Z3-based pipeline showing 70% of proofs are invalid (74% including partial), with audited valid rates of ∼26% on the full test set and ∼13% on the hard-tier subset, against a reported 82.8% (Section 3).
2. A certificate-based intervention using Knuckledragger that prevents assumed-conclusion failures by construction (2% of Z3-only failures) but does not prevent vacuous proofs (40%), and exposes a deeper failure mode: trivial certificates (Section 4).
3. A failure taxonomy and the hypothesis that theorem decomposition is a key bottleneck, with concrete examples from FormalMATH; we note this hypothesis is not yet definitively isolated from other confounds (Section 5).

2 Methodology

We employed three complementary audit approaches, each targeting a different failure surface.

Manual audit. The first author manually inspected 23 Z3 proof modules generated by an LLM pipeline (gpt-5.4-mini, 3 retries) on the 35-problem miniF2F-hard tier (AIME and IMO problems). Each module was classified as Valid (faithful encoding + complete derivation), Partial (correct encoding but incomplete derivation), or Invalid (unfaithful encoding or fundamentally unsound reasoning).

Table 1: Two audits of the Z3-only pipeline on miniF2F. Both show the majority of "proved" results are invalid.

	Manual (23 hard)		**Automated (50 full)**
Verdict	Count	%	%
VALID	3	13%	26%
PARTIAL	8	35%	4%
INVALID	12	52%	70%

Semi-automated vacuity testing. We sampled 50 "proved" results from the full 244-problem miniF2F test set (Z3-only pipeline, 202/244 reported proved). For each proof, we ran automated structural checks: (1) remove the conclusion and re-check—if still `unsat`, the proof is vacuous; (2) replace the conclusion with a random false statement—if still `unsat`, the premises are contradictory; (3) check for trivial arithmetic tails; (4) check for crashes masked by error handling. Final classification was performed by an LLM classifier (gpt-5.4-mini) based on execution traces—categories requiring qualitative judgment (PARTIAL, WRONG_ENCODING, VALID) were reviewed but not independently verified by a human expert. Results are organized into seven categories.

Certificate classification. We ran 54 "proved" FormalMATH results through Knuckledragger with SymPy pre-computation and manually classified the resulting certificates into six categories: VALID (certificate proves the stated theorem), TRIVIAL_CERTIFICATE (certificate proves something unrelated to the theorem, e.g., `x <= x`), PARTIAL (certificate covers part of the theorem), NUMERICAL_ONLY (certificate verifies only a numerical answer without the reasoning), CRASH (pipeline error), and WRONG (certificate proves a false or different statement).

3 The Audit: Z3-Only Pipeline

3.1 Headline Numbers

On the full 244-problem miniF2F test set, our Z3-only pipeline (gpt-5.4-mini, 3 retries) reported 202/244 problems proved, a solve rate of 82.8%. The two audits tell a different story.

The manual audit found 3 of 23 hard-tier proofs genuinely valid (13%). The automated audit found 13 of 50 sampled proofs valid (26%)—the higher rate reflecting that easy-tier problems are more likely to have correct encodings. These numbers bracket the audited valid rate by difficulty tier: approximately 26% on the full test set and 13% on the hard-tier subset, against a reported 82.8%. We do not report a single combined estimate because mixing difficulty tiers into one number obscures the tier-level breakdown.

3.2 Failure Taxonomy

The automated audit revealed seven distinct failure modes, summarized in Table 2.

The dominant failure mode is *vacuous encoding* (40%): the proof-generating model constructs premises that are themselves contradictory, so Z3 returns `unsat` regardless of the conclusion. This is followed by crashes masked by error handling (10%), trivial arithmetic tails (10%), and wrong encodings (8%).

Table 2: Failure taxonomy from the automated 50-sample audit (Z3-only, full miniF2F).

Category	Description	%
Valid	Faithful encoding, complete derivation	26
Vacuous	Premises contradictory; UNSAT is trivial	40
Crash	Z3 timeout/error masked by error handling	10
Trivial tail	Only final arithmetic checked, reasoning skipped	10
Wrong encoding	Mathematical claim not faithfully represented	8
Partial	Correct encoding but incomplete derivation	4
Assumed conclusion	Conclusion asserted as premise	2

3.3 Illustrative Examples

Vacuous encoding (`induction_pprime_pdvdapowpma`). The most common failure. The proof-generating model adds constraints that, taken together, are contradictory, so Z3 returns `unsat` regardless of the conclusion. From our audit: the model encodes a primality constraint that is already UNSAT with the other assumptions before any conclusion is added. Classifier verdict: "The solver constraints are already UNSAT even without the negated claim, so the proof is vacuous."

```
from z3 import *
s = Solver()
p, a = Ints('p a')
s.add(p > 1)
s.add(ForAll([k], Implies(And(k > 1, k < p), p % k != 0)))
s.add(a % p == 0)   # <-- combined with ForAll above,
s.add(a > 0)        #     already UNSAT; negation is irrelevant
s.add(Not(p % a == 0))
assert s.check() == unsat  # vacuous
```

Listing 1: Vacuous encoding: assumptions are already contradictory (schematic from `induction_pprime_pdvdapowpma`).

Assumed conclusion (`amc12a_2002_p6`, `mathd_numbertheory_427`). The conclusion is directly asserted as a constraint. From our audit of `amc12a_2002_p6`: "The code directly encodes the conclusion that every positive `m` has a witness `n`... by explicitly asserting the existence of such `n` as a constraint rather than deriving it."

```
s = Solver()
m, n = Ints('m n')
s.add(m > 0)
# The theorem asks us to prove there exists n s.t. n^2 > m
s.add(n > 0)
s.add(n * n > m)    # <-- this IS the conclusion; it's axiomatized!
s.add(Not(n * n > m))
assert s.check() == unsat
```

Listing 2: Assumed conclusion: target existence claim added as axiom (`amc12a_2002_p6`).

Trivial arithmetic tail (`amc12a_2013_p4`, `mathd_numbertheory_328`). The model skips the mathematical reasoning and checks only that a concrete numerical result satisfies itself. From our audit of `amc12a_2013_p4`: "The code does not encode the actual problem's exponent expression; it directly sets the value." For `mathd_numbertheory_328`: "only performs direct modular arithmetic evaluations with Python's `pow` and checks a hardcoded result."

```
# After extensive comments describing the "reasoning"...
answer = 12   # the intended final answer
s = Solver()
s.add(Not(answer == 12))
assert s.check() == unsat  # trivially: 12 != 12 is unsat
```

Listing 3: Trivial tail: skips reasoning, checks hardcoded answer = answer (`amc12a_2013_p4`).

3.4 Why Z3 Cannot Prevent These Failures

Z3's Python API makes no distinction between axioms and derived facts. Every `s.add()` call has the same status. The solver answers "is this set of constraints satisfiable?" without tracking provenance. A correct Z3 proof requires discipline external to the solver—discipline that LLMs systematically lack.

4 Do Proof Certificates Fix It?

To address the trust gap, we introduced Knuckledragger [9], a Python proof assistant built on Z3 by Philip Zucker. Its key property is a tamper-proof `Proof` type: you cannot construct a `Proof` object without the kernel verifying the derivation. Specifically, `kd.lemma(claim, by=[p1, p2, ...])` asks Z3 to derive `claim` from existing proofs and returns a `Proof` only if successful. Combined with SymPy for algebraic pre-computation, this forms our certification pipeline.

4.1 What Certificates Fix

Certificates prevent one class of failure by construction: *assumed-conclusion* failures, where the proof script directly asserts the conclusion as a premise. In Knuckledragger, every claim must be derived from prior `Proof` objects via `kd.lemma` or `kd.prove`; you cannot introduce an unproven claim as an axiom without explicitly marking it `kd.axiom`. This eliminates the 2% of Z3-only failures classified as ASSUMED_CONCLUSION.

However, certificates do *not* eliminate vacuous proofs. A vacuous proof arises from contradictory premises—the LLM encodes constraints that are already mutually unsatisfiable, so Z3 returns `unsat` regardless of the conclusion. Under Knuckledragger, the LLM can still add contradictory constraints; the kernel only checks that each `kd.lemma` call is derivable from its stated hypotheses, not that those hypotheses are jointly consistent. The 40% of Z3-only failures that are vacuous remain a live failure mode under certification.

On miniF2F, the certificate pipeline shows measurable improvement on solve rate:

- **Knuckledragger v1** (hard 35): 48.6% proved, 37.1% certified.
- **Knuckledragger + Sonnet** (full 244): 100% proved, 89.3% certified.
- **Manual re-execution**: 7/10 sampled certified proofs pass on re-execution (70%).

These numbers look encouraging. But the certificate rate is not the same as the *valid* rate.

4.2 What Certificates Do Not Fix: The FormalMATH Audit

We ran 54 "proved" FormalMATH results through the Knuckledragger pipeline and manually classified the resulting certificates. The results, shown in Table 3, are sobering.

Table 3: Certificate audit of 54 FormalMATH "proved" results (Knuckledragger + SymPy pipeline).

Category	Count	%
TRIVIAL_CERTIFICATE	25	46.3
PARTIAL	9	16.7
NUMERICAL_ONLY	9	16.7
CRASH	4	7.4
VALID	4	7.4
WRONG	3	5.6

Table 4: What `kd.prove` certifies vs. what the problem asks. All three certificates are valid `Proof` objects—but none proves the theorem.

Problem	What the theorem asks	What kd.prove certifies
FormalMATH #12	For all $n > 0$, $\sum_{k=1}^{n} k^2 = n(n+1)(2n+1)/6$	`kd.prove(x <= x)` — reflexivity of $\leq$
FormalMATH #31	If p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$	`kd.prove(Or(True, False))` — tautology
FormalMATH #47	$\sqrt{2}$ is irrational	`kd.prove(2 > 0)` — positivity of 2

Only 4 of 54 certificates (7.4%) are genuinely valid—they prove the stated theorem. The dominant failure is TRIVIAL_CERTIFICATE at 46.3%: the LLM produces a valid `Proof` object, but the certified claim is trivial and unrelated to the theorem. The certificate is real; it just does not prove anything interesting.

4.3 Example: What Certificates Actually Prove

Table 4 shows three concrete examples of the gap between what the problem asks and what the certificate proves.

In each case, the LLM generates a proof script that calls `kd.prove` on a trivially true statement. Knuckledragger correctly certifies the trivial claim—because the claim *is* true. But the certified claim has no logical connection to the original theorem. The LLM has substituted an easy problem for a hard one.

4.4 The Inflation Pattern Is Consistent

Table 5 summarizes the inflation pattern across all three settings. Regardless of backend, benchmark, or certification mechanism, reported rates are 4–10× higher than audited valid rates.

5 The Decomposition Bottleneck

The certificate audit reveals a pattern in our pipeline: the proof-generating model does not decompose theorems. It generates proof scripts that *look* like multi-step proofs—with comments, lemma names, and structured reasoning—but the actual certified claims are trivial. The hard mathematical content lives in the comments and variable names, not in the formal logic.

Consider a typical FormalMATH proof script for "$\sqrt{2}$ is irrational." The LLM generates:

```
import kdrag as kd
# Step 1: Assume sqrt(2) = p/q in lowest terms
```

Table 5: Reported vs. audited valid rates. The two benchmarks use different pipelines and preprocessing; rates are not directly comparable across rows. Within each row, reported and audited rates are comparable.

Benchmark	**Backend**	**Tier**	**Reported**	**Audited valid**
miniF2F (Z3-only)	Raw Z3	Full	82.8%	∼26%
miniF2F (Z3-only)	Raw Z3	Hard	82.8%	∼13%
FormalMATH (KD)[†]	kdrag+SymPy	Mixed	46.7% cert	7.4%

[†] Different benchmark, different pipeline; not comparable to miniF2F rows.

```
# Step 2: Then 2*q^2 = p^2, so p is even
# Step 3: Write p = 2k, then 2*q^2 = 4*k^2, so q^2 = 2*k^2
# Step 4: So q is even, contradicting lowest terms
# Therefore sqrt(2) is irrational.
proof = kd.prove(2 > 0)  # <-- this is all that's certified
```

Listing 4: Typical LLM-generated "proof" of irrationality of $\sqrt{2}$.

The comments describe a correct informal proof. But the formal content—the only thing the kernel checks—is `2 > 0`. The LLM knows what the proof *should* look like. It cannot translate that knowledge into formal sub-lemmas.

This is the decomposition bottleneck: to prove a theorem formally, you must break it into a chain of lemmas where each lemma is within the solver's reach. This requires understanding which steps are trivial for Z3 (arithmetic, propositional logic) and which require careful encoding (induction, case analysis, number-theoretic arguments). Our pipeline treats the proof as a natural-language narrative and appends a trivial formal claim at the end.

Implications. If this pattern holds more broadly, improving the proof backend (better solvers, better certificate mechanisms) has diminishing returns. The binding constraint would be the model's ability to perform multi-step mathematical reasoning that maps to formal sub-lemmas. This is a fundamentally different capability from generating plausible proof narratives. We present this as a hypothesis supported by our audit data, not a definitive finding—other pipelines, models, or prompting strategies may exhibit different behavior.

6 Related Work

Lean-based theorem proving. The dominant paradigm uses Lean 4 with kernel-checked proofs. Seed-Prover [2] combines MCTS with learned value functions; HILBERT [3] uses similar search with improved training. Kimina-Prover [4] and Goedel-Prover-V2 [5] explore reinforcement learning. These systems benefit from Lean's kernel guarantee: if the proof type-checks, it is correct. Our audit demonstrates why this guarantee matters.

SMT-based approaches. MiniF2F-Dafny [7] (POPL 2026) translates problems to Dafny at 55.7%. Dafny's verification condition generation provides stronger guarantees than raw Z3 but weaker than Lean's kernel. Our findings suggest that any SMT-based approach is vulnerable to encoding failures unless independently audited.

Proof integrity. AlphaProof [8] (Nature 2025) notably chose Lean kernel verification over solver-based approaches for IMO problems. Our audit validates this design choice. FormalMATH [6]

Table 6: Comparison with published systems on miniF2F (test split). *Lean kernel provides built-in certification; reported rates *are* certificate rates. †Audited valid rate, not raw certificate rate.

System	**Backend**	**Ref.**	**Rate (%)**
Seed-Prover	Lean 4*	[2]	∼100
HILBERT	Lean 4*	[3]	99.2
Kimina-Prover	Lean 4*	[4]	92.2
Goedel-Prover-V2	Lean 4*	[5]	90.4
AlphaProof	Lean 4*	[8]	—
MiniF2F-Dafny	Dafny/Z3	[7]	55.7
Ours (reported, Z3-only)	Raw Z3		82.8
Ours (audited valid, full)†	Raw Z3		∼26
Ours (audited valid, hard)†	Raw Z3		∼13

extends formalization to broader domains but, as our certificate audit shows, formalization alone does not guarantee proof validity.

LLM evaluation skepticism. The PAJAMA study [11] and "Time to Impeach LLM-as-a-Judge" critique [12] demonstrated systematic over-reporting in LLM evaluation. Work on debate-based verification [13] similarly highlights gaps between perceived and actual quality. Our findings extend this skepticism to formal verification: even in a domain where correctness is supposedly checkable, unchecked results are unreliable.

Knuckledragger. Developed by Philip Zucker [9], Knuckledragger wraps Z3 in a proof-carrying discipline via opaque `Proof` objects. We are, to our knowledge, the first to use it for large-scale LLM theorem proving—and the first to document its limitations when LLMs generate trivial certificates.

7 Discussion

Recommendations. Based on our audits, we make three recommendations for the field:

1. **Always audit.** Report both solve rates and independently audited valid rates. A certificate rate is better than a raw solve rate, but an audited certificate rate is better still.
2. **Report certificate rates AND audit rates.** Certificate rates alone are misleading—our FormalMATH audit shows that 46.7% certificate rate corresponds to 7.4% valid rate. The gap between these numbers is the real trust gap.
3. **Focus on decomposition.** The proof backend is not the bottleneck. Research effort should target LLMs' ability to decompose theorems into non-trivial formal sub-lemmas.

Honest positioning. Table 6 places our results alongside published systems. We are not claiming state-of-the-art performance. We are claiming that honest measurement produces much lower numbers than currently reported for SMT-based approaches.

What certificates get right. We do not want to dismiss certificates entirely. Knuckledragger certificates eliminate an entire class of failures—vacuous proofs—that constitutes 40% of Z3-only

invalidity. This is a genuine and important improvement. The problem is that certificates are necessary but not sufficient: they catch one failure mode while LLMs adapt by producing a different one (trivial certificates).

Limitations. Our manual audits were conducted by a single author with no independent verification. The semi-automated audit uses an LLM classifier (gpt-5.4-mini) for categories requiring qualitative judgment; misclassification is possible and we have not estimated the error rate. The FormalMATH certificate audit covers 54 problems, a small and potentially non-representative sample. We have not audited other systems' proof files—all audits are of proofs generated by our own pipeline, so we cannot directly claim the same failure rates apply to other Z3-based approaches. Knuckledragger relies on Z3's proof-producing mode, which is sound but incomplete—some valid proofs may fail to certify. Finally, our findings apply to Z3-based approaches; Lean-based systems with kernel checking are not subject to the same critique.

Reproducibility. All experiments used gpt-5.4-mini (temperature 1.0, top_p 1.0, up to 3 retries, 60-second Z3 timeout per problem). The audit classification pipeline used gpt-5.4-mini as classifier (temperature 0, structured output mode). Audit results and all generated proof scripts are released at https://github.com/visnkmr/arbiter.

The decomposition problem as a hypothesis. Our audits converge on a consistent pattern: the hard problem in our Z3-based pipeline is not proof verification but theorem decomposition. We present this as a hypothesis—our data supports it but does not establish it as a general law of LLM theorem proving. Until this is tested across multiple models and pipelines, claims about what "LLMs" can or cannot do should be interpreted narrowly. What we can say with confidence is that this specific pipeline, on these benchmarks, fails primarily at decomposition. The field would benefit from benchmarks and methods that specifically target decomposition, not just end-to-end solve rates.

Acknowledgments

We thank Philip Zucker for developing Knuckledragger and for discussions about proof-carrying code in SMT solvers.

References

[1] K. Zheng, J. M. Han, and S. Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.

[2] Seed-Prover Team. Seed-Prover: Large language models as formal theorem provers via MCTS. *arXiv preprint arXiv:2507.23726*, 2025.

[3] Apple ML Research. HILBERT: A large language model approach to formal theorem proving. *arXiv preprint arXiv:2509.22819*, 2025.

[4] Kimina Team. Kimina-Prover: Scaling formal theorem proving with reinforcement learning. *arXiv preprint*, 2025.

[5] Goedel Team. Goedel-Prover-V2: Improved neural theorem proving via synthetic data and curriculum learning. *arXiv preprint*, 2025.

[6] FormalMATH Team. FormalMATH: Benchmarking formal mathematical reasoning of large language models. *arXiv preprint arXiv:2505.02735*, 2025.

[7] Y. Sun, Y. Wu, and X. Si. Formal verification of LLM-generated mathematical proofs via Dafny. In *Proceedings of the ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 2026. *arXiv preprint arXiv:2512.10187*.

[8] DeepMind AlphaProof Team. AlphaProof: AI achieves silver-medal level performance at the International Mathematical Olympiad. *Nature*, 2025.

[9] P. Zucker. Knuckledragger: A Python proof assistant built on Z3. https://github.com/philzook58/knuckledragger, 2026.

[10] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.

[11] PAJAMA Authors. PAJAMA: Systematic evaluation of LLM-as-a-judge. *arXiv preprint*, 2025.

[12] M. Chen and colleagues. Time to impeach LLM-as-a-judge: Systematic biases in automated evaluation. *arXiv preprint*, 2025.

[13] N. Kenton, T. Everitt, and J. Leike. Debate does not (yet) help humans identify deceptive AI. In *International Conference on Learning Representations (ICLR)*, 2025.